指令设计与功能验证报告

等一个棋迹（GEMwhu） 武汉大学集成电路学院

本阶段面向香山昆明湖 V2 定义了固定八项 int8 有符号点积指令（vdot.vv），将其挂载至向量执行通道 VFEX0，内部采用 19 位无损累加树与两级运算流水。验证工作通过综合测试程序对 939 组输入执行全 128 位寄存器比对，涵盖正数、含负数及边界极值，并在基础材料中归档了 16 个功能测试程序的 16 次整核仿真记录。本册依次介绍指令架构语义、Chisel 核心源码、流水线接入改动以及功能验证方案。

赛题要求	本册对应章节与内容
2.1 指令设计	第 4 章运算语义与指令编码，第 5 章流水线微架构与后端接入
2.2 RTL 实现	第 5 章核心 Chisel 源码解析及香山后端改动清单
2.3 正数 负数 边界	第 6 章测试向量组织、整核回归程序及实测结果汇总

4 vdot.vv 指令设计

4.1 运算语义

vdot.vv 是本次为昆明湖 V2 处理器定制的固定八项有符号字节向量点积指令。指令执行时提取两个源向量寄存器（vs1、vs2）的低 64 位，将其视作 8 对有符号 int8 元素相乘，8 路乘积从 0 开始累加，所得精确和经符号扩展后写入目的寄存器 vd 的低 32 位，同时将 vd 的高 96 位强制清零。源寄存器的高 64 位（高 8 字节）不参与运算，执行过程也不读取目的寄存器的历史旧值。

```
sum = Σ signed8(vs1[i]) × signed8(vs2[i]),  i = 0…7
vd[31:0]   = sign_extend_32(sum)
vd[127:32] = 0
```

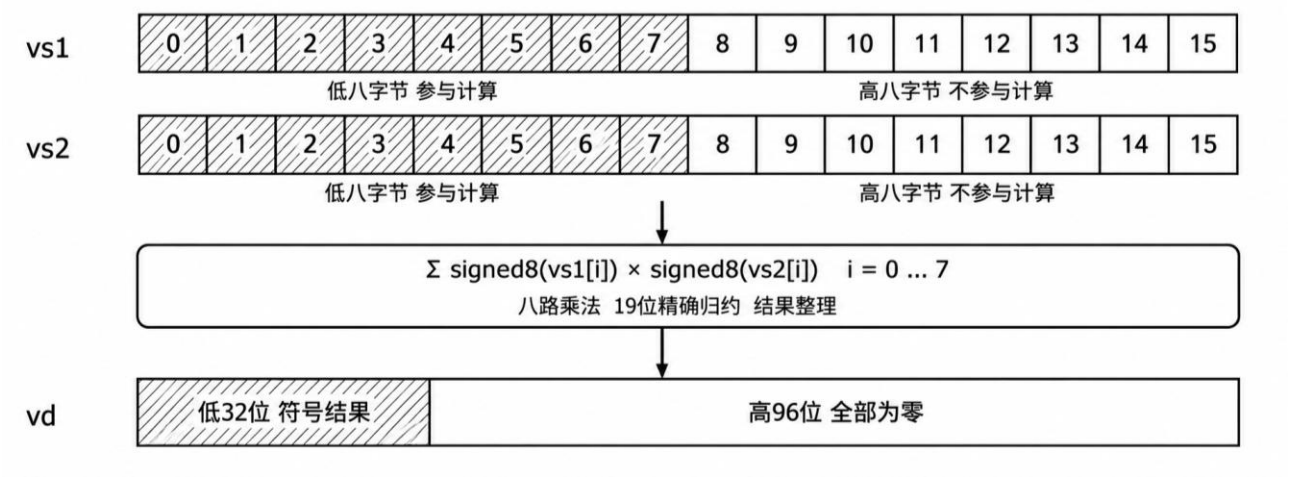


图 4-1 固定八项点积的数据通路示意：仅取两个源寄存器的低 8 字节参与乘加，累加结果在目的寄存器低 32 位做符号扩展，高 96 位清零。

举例而言，当点积计算值为 −8 时，写回目的寄存器的 128 位完整内容为 0x000000000000000000000000fffffff8。这里符号扩展严格约束在低 32 位字长内，高 96 位保持全零而不会填入符号位。

架构配置项	本设计具体约定
VLEN、SEW、LMUL	固定支持 VLEN=128 位、SEW=e8、LMUL=m1
vm、vstart、vill	要求 vm=1、vstart=0、vill=0；不支持掩码执行
向量长度 VL	固定提取低 8 字节计算，不受 VL 截断影响；已实测 VL=0、1、4、8、16
尾部与高位策略	整寄存器覆写：低 32 位写符号扩展结果，高 96 位无条件清零
寄存器重名（Alias）	支持目的与源寄存器重合，已验证 vd=vs1、vd=vs2、vd=v0
非法配置响应	统一触发非法指令异常（Illegal Instruction），详见第 6 章验证

需要说明的是，本设计将固定计算低八字节作为硬性指令语义，即使当前向量长度配置为 VL=0 也照常完成八项点积并写回。在编写低 VL（如 VL<8）的测试程序时，必须先在 VL≥8 状态下通过标准向量加载指令把 8 字节源数据完整装入寄存器，随后再用 vsetvli 切换至待测的低 VL 执行 vdot.vv，否则前端的向量加载指令自身就会因 VL 截断而漏载源数据。

4.2 指令编码与软件调用

位域范围	字段名称	编码取值与说明
31:26	funct6	101100
25	vm	1（固定无掩码）
24:20	vs2	第二源向量寄存器编号
19:15	vs1	第一源向量寄存器编号
14:12	funct3	010（对应 OPMVV 向量-向量算术类）
11:7	vd	目的向量寄存器编号
6:0	opcode	1010111（0x57，RVV 向量主操作码）

由于上游标准 GNU 汇编器尚未内置 vdot.vv 助记符，测试程序直接通过 .word 伪指令嵌入 32 位机器码（如 vdot.vv v4,v9,v8 写作 0xb2942257）。包含机器码调用与 128 位全寄存器校验的最小工作示例见 examples/minimal_vdot.S。

该指令编码借用了 RVV 1.0 规范中 OPMVV 格式下的一处未分配编码点，与 RISC-V 社区公开的 Zvqdotq 扩展草案中的 vqdot.vv 存在编码重合。但二者在架构行为上截然不同：本赛题指令在 SEW=e8 下将低 8 字节规约为单个 32 位和并覆写整个目的寄存器，而 Zvqdotq 草案则要求在 SEW=e32 下按每 4 字节一组执行四路乘累加，需读取目的寄存器旧值。当前硬件仅实现了赛题规定的固定八项语义，未做双语义分流，因此不能与 Zvqdotq 软件生态直接混用。

若后续面向生产环境演进，可将指令编码迁移至 RISC-V 预留的 custom 自定义编码空间，或者彻底对齐 Zvqdotq 草案重构乘累加数据通路。

4.3 位宽推导

从算术位宽来看，单个 int8 操作数的取值区间为 [−128, 127]，单路乘法结果为 [−16256, 16384]，8 路乘积求和后的极值范围扩展至 [−130048, 131072]。由于 18 位二进制补码所能表达的最大正数仅为 131071（恰好比八个 −128 自乘累加的峰值 131072 少 1），要想在全输入范围内彻底消除累加溢出，中间加法树至少需要保留 19 位有符号精度，最后再向 32 位结果域做符号扩展。

流水处理阶段	数据形态与位宽设计
源操作数提取	从 vs1、vs2 低 64 位各取 8 个有符号 int8
第一级：并行乘法	8 路并行有符号乘法，单路 int16 乘积符号扩展至 19 位寄存
第二级：归约加法树	平衡二叉树累加为 19 位精确有符号和，无损覆盖八项极值边界
结果拼接与写回	19 位和符号扩展至低 32 位，高 96 位补零，凑齐 128 位写回

5 硬件实现与香山接入

5.1 两级数据通路

硬件实现上，新增的 VDotU 功能单元挂载在香山后端的向量执行通道 VFEX0 下，直接继承 VecPipedFuncUnit 基类，从而无缝接入香山原有的流水级有效位握手、异常向量传递及 128 位向量写回总线。指令经前端译码与重命名后进入向量发射队列，待两个源向量寄存器就绪即派发至 VDotU，计算结果沿既有写回网络更新物理寄存器并由 ROB 按序提交。

VDotU 内部算术通路划分为两级流水：第一级（S1）截取源向量低 8 字节，并行完成 8 路有符号 8×8 乘法并将乘积锁存；第二级（S2）通过平衡加法树完成归约求和与高位清零拼接。后端配置中声明该单元为固定 2 周期执行延迟。对于无数据相关的独立 vdot.vv 序列，发射队列可每周期连续送入一条指令；但在存在写后读相关的依赖链中，后继指令还需等待写回总线传输、唤醒及发射调度，因此端到端的依赖链发射间隔会明显大于单元内部的 2 周期算术延迟。

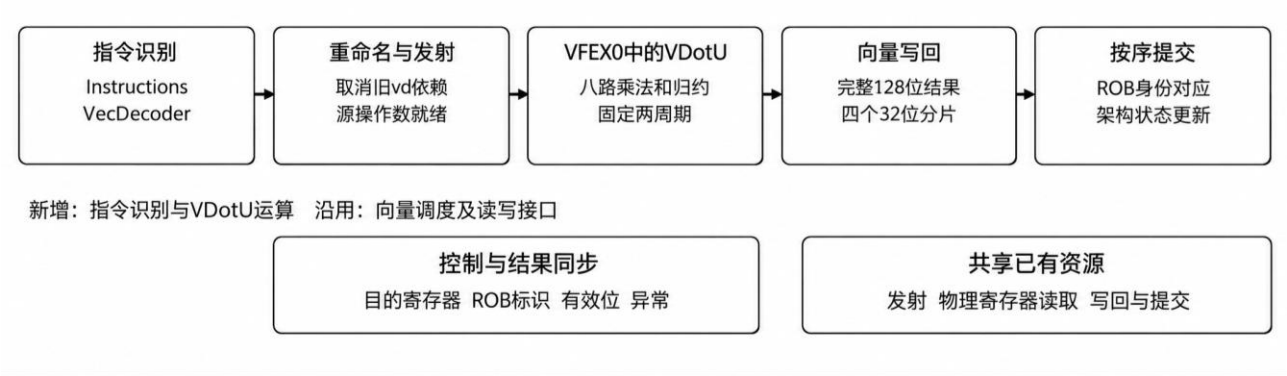


图 5-1 VDotU 在香山向量后端中的集成位置。核心算术逻辑位于 VFEX0 通道内，指令发射、物理寄存器读写及 ROB 提交均复用香山既有基础设施。

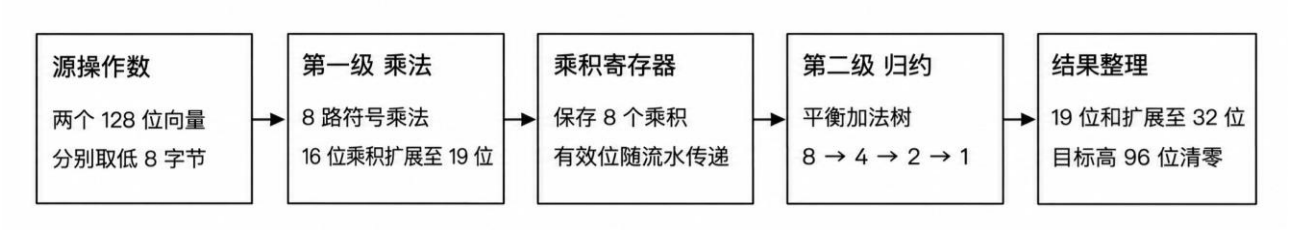


图 5-2　基于 VDotUnit.scala 实装参数绘制的两级算术流水结构。仅低 8 字节进入实际乘法阵列，生成器预留的高位通道在硬件上恒定接零。

以正数测试集的第 1 组样本为例：当两个源向量的低 8 字节均设为 1、2、3、4、5、6、7、8 时，第一级乘法器并行算出 1、4、9、16、25、36、49、64，第二级加法树求和得到 204（0xcc），最终写入目的寄存器的 128 位值为 0x000000000000000000000000000000cc。

5.2 关键源码

图 5-3 与图 5-4 直接截取自本次提交的 RTL 源码与补丁脚本（左侧保留原文件行号）：图 5-3 展示了有符号乘法、级间寄存及 128 位输出拼接逻辑；图 5-4 则对应译码阶段解除旧目的寄存器依赖以及输出端的非法配置异常判定。完整的 Chisel 与脚本源码均以纯文本形式保存在证明材料目录中。

VDotUnit.scala 有符号乘法和第一级寄存

```
54      private val prodComb = VecInit(Seq.tabulate(p.lanes) { i =>
55        val prod = vs1g(i).asSInt * vs2g(i).asSInt
56        prod.pad(p.accw).asUInt
57      })
58
59      // Stage one holds sign-extended products only when a new input is present.
60      private val prod = RegInit(VecInit(Seq.fill(p.lanes)(0.U(p.accw.W))))
61      private val fireS1 = RegInit(false.B)
62      when(io.fire) { prod := prodComb }
63      fireS1 := io.fire
```

VDotUnit.scala 结果格式与第二级有效位

```
98      private val red = RegInit(0.U(p.accw.W))
99      private val validS1 = RegInit(false.B)
100     red := redComb
101     validS1 := fireS1
102
103     private val outField: UInt =
104       if (p.outw > p.accw)
105         Cat(Fill(p.outw - p.accw, red(p.accw - 1)), red).asUInt
106       else
107         red(p.outw - 1, 0)
108
109     // Sign-extend only the low result word; every other destination bit is zero.
110     io.vd := Cat(0.U((p.vlen - p.outw).W), outField).asUInt
111     io.valid := Mux(reset.asBool, false.B, validS1)
```

图 5-3 VDotUnit.scala 第 54~63 行与第 98~111 行源码。乘积按有符号数扩展至累加位宽，最终仅对低 32 位做符号扩展并将高 96 位清零。

install.sh 取消旧目的寄存器读取

```
244     when (decodedInst.fuType === FuType.vdot.U) {
245       decodedInst.srcType(2) := SrcType.no
246       decodedInst.vpu.isDependOldVd := false.B
247       decodedInst.vpu.isWritePartVd := false.B
248     }
```

VDotU.scala 使用输出阶段配置检查异常

```
74      private val sewIllegal = outVecCtrl.vsew =/= VSew.e8
75      private val lmulIllegal = outVecCtrl.vlmul =/= 0.U
76      private val maskIllegal = !outVm
77      private val restartIllegal = outVstart =/= 0.U
78      io.out.bits.ctrl.exceptionVec.get(ExceptionNO.illegalInstr) :=
79        outVecCtrl.vill || sewIllegal || lmulIllegal || maskIllegal || restartIllegal
```

图 5-4 install.sh 第 244~248 行与 VDotU.scala 第 74~79 行源码。译码级切断旧 vd 读依赖，执行级利用流水对齐后的控制字判定配置异常。

源码位置：rtl/xiangshan-patch/VDotU.scala 与 rtl/chisel/vdot/vdot/src/vdot/VDotUnit.scala。自动化集成脚本位于 rtl/xiangshan-patch/apply.sh，经 Chisel 编译生成的 SystemVerilog 模块保存在 rtl/xs_wrapper_generated/VDotU.sv。

5.3 控制与异常

控制通路上，译码器将 vdot.vv 标记为整寄存器覆写指令并解除对旧 vd 的读依赖；VDotU 在结果输出级对 SEW=e8、LMUL=m1、无掩码（vm=1）、vstart=0 及 vill=0 等合法条件做统一核验，一旦违规即拉高非法指令异常标志并随计算结果同步送回后端。在整核运行中，VFEX0 通道上的其他向量指令仍会与 vdot.vv 共享发射端口与写回总线。

尽管底层算术生成器 VDotUnit 采用了参数化编写方式，但在本次面向昆明湖 V2 的实例化配置中，高 8 路归约输入已在硬件层直接绑零，仅低 8 路字节乘法器实际生效。因此阅读源码时需区分参数化 Chisel 模板与本次整核固化的硬件实例，其他 SEW 位宽或多通道配置尚未在整核层面展开实现与验证。

5.4 接入改动与设计选择

后端改动环节	涉及源码文件	具体功能职责
指令位串识别	Instructions.scala、VecDecoder.scala	匹配 VDOT_VV 机器码并将功能单元类型路由至 FuType.vdot
单元类型与属性	FuType.scala、FuConfig.scala	注册向量整数单元属性，声明固定 2 周期延迟与 128 位写回宽度
执行通道挂载	Parameters.scala	在向量执行通道 VFEX0 的功能单元列表中挂载 VdotCfg
解除旧目的依赖	DecodeUnit.scala	将 srcType(2) 设为 no，并将 isDependOldVd 与 isWritePartVd 置假
非法配置拦截	VecExceptionGen.scala、VDotU.scala	核验 SEW/LMUL/vm/vstart/vill 合法性，异常标志随流水输出同步
算术核与接口封装	VDotUnit.scala、VDotU.scala	实现两级八项有符号点积通路，对接香山 VecPipedFuncUnit 握手接口

表列香山源码文件均位于 src/main/scala/xiangshan 对应子模块下；实际代码注入由 rtl/xiangshan-patch/install.sh 自动完成（apply.sh 为兼容调用入口）。需要注意，源码树中的 VdotCfg.scala 仅保留了开源许可证头注释，真正的 FuConfig 单元定义是由 install.sh 直接写入 FuConfig.scala 的。

将并行乘法与加法归约切分为两级流水，使单级逻辑深度可控并维持固定的 2 周期算术延迟。第二级采用平衡二叉树结构，将 8 路 19 位扩展乘积按 8→4→2→1 逐层相加，确保极值输入下不发生中间截断溢出。这种切分引入了一级乘积寄存器与一级结果寄存器；当后继指令依赖当前 vdot 结果时，除这 2 周期算术延迟外，还需经历写回通路、发射唤醒与重发射过程。第三阶段分册第 10 章的实测波形证实内部运算为 2 周期、RAW 依赖链发射间隔为 7 周期。

在译码阶段将 srcType(2) 设为 SrcType.no 并清零 isDependOldVd，源于 vdot.vv 总是完整改写 128 位目的寄存器（低 32 位写结果、高 96 位清零）。既然无需保留目的寄存器的任何历史位，硬件上便不存在对旧 vd 的真实数据依赖。这一处理不仅消除了不必要的伪依赖等待，还保证当目的寄存器与源寄存器重名（如 vd=vs1）时，指令依然能安全读取重命名后的源物理寄存器完成计算。

6 功能验证

6.1 用例规模与验证方法

综合验证集共构造了 939 组测试向量，由 46 组正数用例、48 组含负数用例、45 组边界极值用例以及 800 组固定随机种子用例组成；若按两个 128 位源寄存器的完整取值去重，共对应 936 组独立输入模式。此外，针对向量长度与寄存器重名的专项测试程序直接从上述 939 组样本中抽样复用了 171 组，统计总用例规模时不再重复累加。

具体构成上，46 组正数样本与 48 组含负数样本分别包含 6 组、8 组手工构造的典型激励及各自 40 组受控随机激励；45 组边界样本则由 15 组手工极值用例与 30 组边界位置组合构成，外加 800 组全 int8 范围随机激励。上述分类旨在系统覆盖各类算术角落情况，在整核回归时统一打包进综合镜像运行，而非拆成互不相交的零散批次。

为严格检验高位清零特性，每组用例执行前均先把目的寄存器填满非零垃圾值，待 vdot.vv 执行完毕后将 128 位结果完整转储至内存，同时比对低 32 位算术和与高 96 位零值。整核综合测试程序 corpus_vl8_ta1 在单次运行中即可跑完上述全部 939 组激励，正数、负数与边界子集通过各自的标签清单进行分类核对。

基础验证材料共归档了 16 个裸机功能测试程序的 16 次整核仿真记录（含 9 个常规算术与合法配置程序、6 个非法配置异常测试程序以及 1 个原生 vadd 对照程序），全部日志保存在 results/cluster/20261019/ 目录下；第三阶段开展的 420 次性能基准测试则单独归档统计。每次整核运行必须同时满足三个条件方判定为 PASS：仿真进程正常退出、DiffTest 全程零失配、以及最终退出的 PC 精确命中所设定的成功地址。

验证维度	测试程序或激励集	整核仿真结果
正数 / 负数 / 边界	cases_pos / neg / bound；corpus_vl8_ta1	46 / 48 / 45 组激励，128 位全寄存器比对通过
全范围随机与综合	cases_rand；corpus_vl8_ta1	800 组随机激励，合计 939 组全部通过
VL 与尾部策略组合	scope_vl0_ta0、vl1_ta1、vl4_ta0、vl8_ta0、vl16_ta1	5 个专项整核程序全部通过
源与目的寄存器重名	alias_v0、alias_v8、alias_v9	3 个重名专项整核程序全部通过
非法向量配置拦截	illegal_sew / lmul / fractional_lmul / mask / restart / vill	6 个异常测试程序精确捕获非法指令异常
原生向量加法对照	vadd_full_register	1 个基线对照程序整核通过

表中的程序名与 programs/目录下的汇编源码（.S）及二进制镜像（.bin）一一对应；各程序的预期结束 PC 与实测运行状态汇总在基础结果目录的 program_manifest 与 summary 文件中。其中 5 个 scope 配置程序与 3 个 alias 重名程序复用了综合集中的 171 组激励。

6.2 代表性算术用例

用例类别	两组源寄存器低 8 字节输入	理论点积和	目的寄存器低 32 位（Hex）
正数常值	全 1 × 全 1（高 8 字节亦填 1）	8	0x00000008
正数递增	1…8 × 1…8	204	0x000000cc
正负相乘	全 −1 × 全 1	−8	0xfffffff8
负向极值下界	全 −128 × 全 127	−130048	0xfffe0400
正向极值上界	全 −128 × 全 −128	131072	0x00020000
零值与高位隔离	低 8 字节全 0，任意改变高 8 字节	0	0x00000000

上表所列算术用例在实测中其目的寄存器高 96 位均严格清零。其中正数首项（全 1 点积）特意将源寄存器的高 8 字节也全部置为 1，实测输出依然为 8，直接验证了高 64 位无关数据被有效屏蔽；边界测试集则重点变换了第 0、第 7 与第 8 字节的极值位置。各分类的详细输入数据与期望标签存放在 tests/ 目录下的 cases_pos.labels.csv、cases_neg.labels.csv 和 cases_bound.labels.csv 中。

特别是正数全 1 样本（高 8 字节同样填 1 而结果保持 8）与上界极值样本（八对 −128 相乘累加得出 131072，对应 0x00020000），分别从硬件层面证实了源寄存器高 64 位隔离与 19 位有符号正极值无溢出，且两者的目的寄存器高 96 位均无任何脏数据残留。

6.3 配置、重合与检查能力

除算术精度外，专项测试集还遍历了 VL=0、1、4、8、16 与不同尾部策略（ta/tu）的组合，并检验了 vd 与 vs1、vs2 或 v0 重合时的重命名正确性；针对 SEW=e16/e32/e64、LMUL≠m1、启用掩码（vm=0）、非零 vstart 以及 vill=1 等非法配置，6 个异常测试程序逐一核对了硬件抛出的非法指令异常码及异常返回 PC。

在模块级与包装层验证中，测试脚本共执行了 96 项定向检查（含 40 项合法配置状态、52 项非法状态阻断及 4 项故障注入识别），并利用 48 组定向激励对 Chisel 算术核、生成的 SystemVerilog 网表与软件黄金模型进行了交叉核对。实验中人为注入的 41 种典型 RTL 实现缺陷全部被测试集成功捕获。至于同时交换两个源向量对应通道的顺序，因在代数上完全等价于点积交换律，故不计入硬件缺陷变异体。

核心验证材料位于：programs/corpus_vl8_ta1.S；results/cluster/20261019/；/verification/check.py 与 coverage.csv。关于 RTL 与 NEMU 动态参考库锁步协同验证的具体机制，详见第三阶段分册第 7 章。